

Predmetni projekat

Računarstvo u oblaku

Kroz predmetni projekat neophodno je implementirati platformu za prikupljanje, procesiranje, čuvanje i analizu podataka sa različitih društvenih mreža i blog portala. Rešenje mora da bude implementirano koristeći **AWS** platformu. Dizajn procesiranja podataka mora da prati [Medalion](#) arhitekturu.

Funkcionalni zahtevi

1. Prikupljanje podataka (bronze layer)

Neophodno je prikupiti podatke sa 2 izvora podataka (datasource): [Hacker News](#) i [X](#) (Twitter).

1.1. Hacker News izvor podataka

Hacker News predstavlja portal za objavljivanje blogova, vesti i komentara na različite teme. Neophodno je dnevnom nivou prikupiti sve objave (story), pitanja (asks), komentare (comments), ponude za poslove (jobs) i ankete (poll) koji su kreirani prethodnog dana. API je besplatan, a dokumentacija je dostupna [ovde](#). Od koristi može biti i [HN Search API](#) koji pretražuje portal na osnovu zadatih ključnih reči.

Prikupljanje podataka treba da se implementira pomoću Lambda funkcije. Funkcija treba da upiše prikupljene podatke u S3 bucket u njihovom izvornom obliku. Nikakvo procesiranje ili transformacija podataka **nije dozvoljena**, jer S3 bucket predstavlja **bronze layer** Data Lake-a koji je namenjen da čuva podatke u njihovom izvornom obliku.

1.2. X (Twitter) izvor podataka

X (Twitter) je društvena [mreža](#) za objavljivanje mini postova. S obzirom na to da je besplatna verzija X API jako limitirana, mogu se iskoristiti već postojeći dataset-ovi na Internetu, ručno formirati ili generisati dataset-ove. Dataset-ove je neophodno ubaciti u Data Lake bucket. Ovu su neki primeri dataset-ova koji se mogu koristiti ali nisu obavezni: [Bitcoin Tweets](#), [Covid Tweets](#).

2. Normalizacija podataka (silver layer)

S obzirom na to da bronze layer Data Lake-a može da sadrži podatke u različitim formatima i same strukture podataka mogu biti različite, neophodno je te podatke svesti na jedan format i formirati odgovarajuću strukturu podataka, odnosno šemu podataka. Bez formirane šeme podataka ne mogu se pisati upiti (query-ji) u kasnijim fazama obrade podataka (upiti se ne mogu pisati na slepo). Ovaj proces se naziva **normalizacija** podataka.

Implementirati Lambda funkciju (ili funkcije) koja će raditi normalizaciju podataka. Normalizacija obuhvata:

- Poravnanje ugnježenih struktura. Na primer *kids* polja u Hacker News objavama.
- Poravnanje vremena. Hacker News koristi Unix Epoch format (1736978058), dok X koristi ISO-8601 (2026-01-15T21:54:18Z). Vreme treba da se poravnava u jedan UTC format.
- Čišćenje vrednosti podataka. Na primer, u nekim Hacker News objavama postoje HTML tagovi (<p>, <i>). Te tagove treba počistiti.
- Uklanjanje duplikata.
- Dodatna procesiranja podataka koje smatrate da su neophodne a nisu prethodno navedene.
- Uspostavljanje šeme (strukture) podataka. Definirati tabele (dataframe-ove) sa njihovim kolanama i relacija između tabela. Po pravilu šema treba da ima što manje redundantnosti i da zadovoljava 3NF. Tabele treba sačuvati u [parquet](#) format i particionirati podatke.

Što se tiče uspostavljanja strukture podataka, ona nije jedinstvena i može se razlikovati u zavisnosti od toga koji podaci su interesantni i imaju benefita. Ova struktura podataka direktno utiče na kasnije faze obrade podataka, i bitno je naglasiti da se može menjati tokom vremena naročito ako su uočeni nedostaci u šemi podataka.

Jedna konkretan primer uspostavljanje strukture podataka bi bio da se šema podataka sastoji od 2 tabele:

- users
 - user_id: UUID, generisani id,
 - username: String, preuzet sa Hacker News i X platforme,
 - platform: String, 'Hacker News' ili 'X',
 - karma_score: Integer, korisnikova reputacija na Hacker News, null za X korisnike,
 - is_verified: Boolean, da li je korisnik verifikovan na X platformi, null za Hacker News korisnike.
 - created_at: Timestamp, normalizovan u UTC ISO-8601 formatu.
- posts
 - post_id: String, originalni id iz Hacker News ili X platforme,

- `author_username`: String, strani ključ ka `users` tabeli,
- `content_text`: String, sadržaj objave, HTML tagovi počišćeni,
- `created_at`: Timestamp, normalizovan u UTC ISO-8601 formatu,
- `post_type`: String, 'story', 'comment', 'tweet', 'retweet'.

Tabela `users` bi se partitionisala po `platform` koloni, dok bi se tabela `posts` partitionisala na onsovu `timestamp` kolone. Data Lake bucket bi u tom slučaju izgledao ovako:

```
s3://social-medias/silver/
├── posts/
│   ├── year=2026/month=01/day=15/
│   │   └── data_001.parquet
├── users/
│   ├── platform=HackerNews/
│   └── platform=X/
```

Za pisanje i čitanje u parquet format može se koristiti [awsranlger](#) biblioteka, kao i njen Lambda [layer](#). Konkretni primer partitionisanja podataka je dostupan [ovde](#).

3. Transformacija podataka (gold layer)

Implementirati Lambda funkciju (ili funkcije) koja transformiše podatke i kreira određene metrike i KPI (Key Performans Indicators).

Izračunati sledeće metrike:

- Koliko se dnevno kreira objava (story), pitanja (asks) komentara (comments), ponuda za posao (jobs), anketa (poll) na Hacker News portalu na dnevnom nivou.
- Broj korisnika sa Hacker News portala na dnevnom nivou.
- Broj korisnika sa X platforme na dnevnom nivou.
- Prvih 10 korisnika X platforme sa najvećim brojem pratilaca.
- Prvih 10 korisnika Hacker News portala sa **najvećim karma score**-om na dnevnom nivou.
- Prvih 10 korisnika Hacker News portala sa **najmanjim karma score**-om na dnevnom nivou.
- Prvih 10 ponuda za posao na Hacker News portalu sa najvećim *score*-om na dnevnom nivou.
- Prvih 10 objava na Hacker News portalu sa najvećim *score*-om na dnevnom nivou.

Izračunati sledeće KPI:

- **Data Quality Score:** Pokazuje procentualno koliko redova tabela (dataframe-ova) nisu null. Ovaj indikator pokazuje koliko je normalizacija podataka dobro urađena.

Za dizajniranje šeme podataka možete koristiti [Star Schema](#).

Na primer za praćene broja korisnika na platformama bi se formirala sledeća tabela:

- **daily_users_metric**
 - **date:** date, datum,
 - **platform:** String, 'Hacker News' ili 'X',
 - **total_users:** Integer, ukupan broj korisnika na određenoj platformi,
 - **new_users:** Integer, broj novih korisnika registrovani za određen dan i platformu.

date	platform	total_users	new_users
2025-01-15	Hacker News	11500	100
2025-01-15	X	456	74
2025-01-16	Hacker News	12030	530
2025-01-16	X	523	87

Particionisanje bi se radilo po *platform* i *date* koloni:

```
s3://social-medias/gold/
├── daily_users_metric/
│   ├── platform=HackerNews/
│   │   ├── date=2026-01-15/
│   │   │   └── data_001.parquet
│   │   ├── date=2026-01-16/
│   │   │   └── data_001.parquet
│   └── platform=X/
│       ├── date=2026-01-15/
│       │   └── data_001.parquet
│       └── date=2026-01-16/
│           └── data_001.parquet
```

4. Vizualizacija podataka

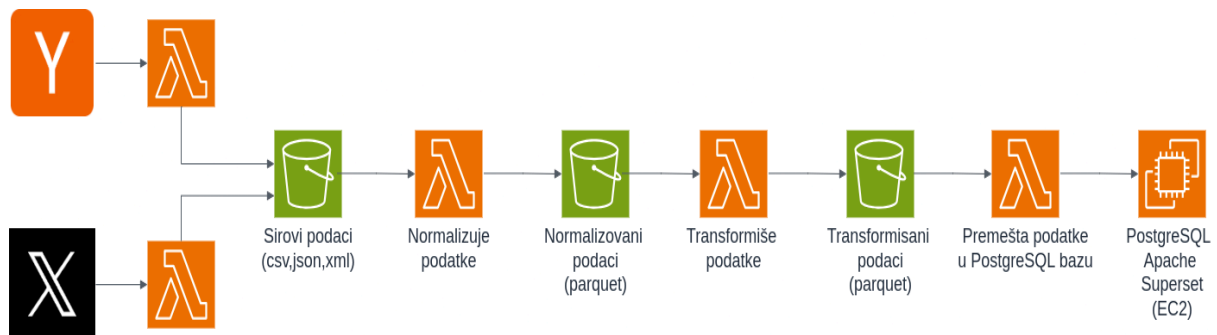
Metrike i KPI koji su nastali transformacijom podataka treba vizualizirati koristeći [Apache Superset](#) alat. S obzirom na to da Apache Superset ne podržava direktno vizualizaciju podataka sa S3 bucket-a u parquet format neophodno je metrike i KPI sačuvati u PostgreSQL bazu. Zatim je neophodno konfigurisati Apache Superset da čita podatke iz PostgreSQL baze. Apache Superset i PostgreSQL treba hostovati na

EC2 instanci. Takođe, neophodno je implementirati Lambda funkciju koja će metrice i KPI iz S3 bucket-a premeštati u PostgreSQL bazu na EC2 instanci.

5. Notifikacije

Neophodno je namestiti notifikacije ka Discord serveru za sve job-ove koji su pali ili su se neuspešno izvršili. Može se koristiti neka druga platforma za notifikacije, nije obavezno koristiti Discord.

Diagram obrade podataka bi bio sledeći:



Napomena: Može se koristiti Step Functions servis kako bi se normalizacija i transformacija podataka razdvojila u više zasebnih koraka, odnosno Lambda funkcija i time pojednostavili implementaciju samih funkcija.

Nefunkcionalni zahtevi

6. Infrastructure as Code (IaC)

Sva infrastruktura mora da bude napisana koristeći neki od IaC alata: [CDK](#), [CloudFormation](#), [Terraform](#) ili [Terragrunt](#).

7. Kontrola mrežne komunikacije

Celokupna infrastruktura treba biti implementirana unutar VPC mreže, uz primenu principa najmanjih privilegija (least privilege). Dozvoljena je isključivo minimalno potrebna mrežna komunikacija između servisa korišćenjem sigurnosnih grupa i mrežnih pravila.

Bodovanje

1. Prikupljanje podataka (bronze layer)	10
2. Normalizacija podataka (silver layer)	14
3. Transformacija podataka (gold layer)	10
4. Vizualizacija podataka	8
5. Notifikacije	5
6. Kontrola mrežne komunikacije	3
Ukupno:	50

Napomena: Infrastructure as Code (IaC) je eliminacioni i projekat koji ne ispunjava ovaj zahtev se neće ni pregledati.

Pravila polaganja

- Projekat se radi u timovima do 3 člana.
- Projekat možete implementirati u bilo kom programskom jeziku i radnom okviru. Ako se odlučite za tehnologiju koja nije pokrivena na vežbama, pomoć u tom slučaju je ograničena.
- Za sve slučajeve koji nisu pokriveni u specifikaciji, studentima se daje mogućnost da ih reše na način koji je njima najprikladniji.
- Projekat se polaže kroz kontrolnu tačku koja će se održati u toku semestra i odbranu koja će se održati u ispitnim rokovima (jednom u junsko-julskom ispitnom roku i jednom u avgustovsko-septembarskom ispitnom roku).